NUMERICAL ALGORITHMS / MPI / HPC

Numerical Inverse Problems & MPI Parallel Computing

Master’s Research Implementation Summary · PhD Application Portfolio

> My primary work is translating mathematical algorithms and pseudocode into Python implementations, developing MPI-parallel versions, and adapting analytical efficiency calculations to the algorithm variants used in my research. I produced the numerical comparisons and presented my own conclusions.

Algorithm implementation and adaptation

Implemented CG-type numerical algorithms in Python and adapted the formulations and cost analysis considered in my May 2026 seminar, including regularization-related terms. Investigated numerical reconstruction for linear inverse problems.

MPI parallelization and communication

Constructed 2-D Cartesian process grids, block-distributed dense matrices, local operations, and row/column communicator reductions. Implemented reusable persistent `Allreduce_init` requests using `Start/Wait`, alongside experiments with nonblocking collectives. This is an implemented strategy, not a proven isolated speedup percentage.

LI WEIZHI / 李伟志

Lomonosov Moscow State University · Faculty of Physics · Mathematics Research Group

Research portfolio · October 2026

Python · MPI / mpi4py · Numerical Linear Algebra · Inverse Problems

1. Research problem and personal responsibility

Discretized linear inverse problems take the form $Ax \approx b\delta$. Iterative methods mainly involve matrix-vector products, adjoint matrix-vector products, dot products and vector updates. MPI implementations must coordinate distributed data storage and global reductions. My main role was building numerical code and carrying out the parallel implementation and comparisons.

2. Distributed-memory implementation and persistent collectives

Parallel design: 2-D Cartesian process grids with matrix blocks assigned to ranks; row and column communicators recover the global quantities needed for the iterative steps. Representative source: `cg_2d_variant_1.py` and `cg_2d_variant_2.py`.

Communication optimization: both implementations contain calls to `Allreduce_init` and reuse prepared collective requests via `Start()` and `Wait()`. This avoids re-describing the same collective call inside each iteration; the net wall-clock benefit still requires isolated benchmarking and inclusion of setup cost.

3. Model-based efficiency comparison

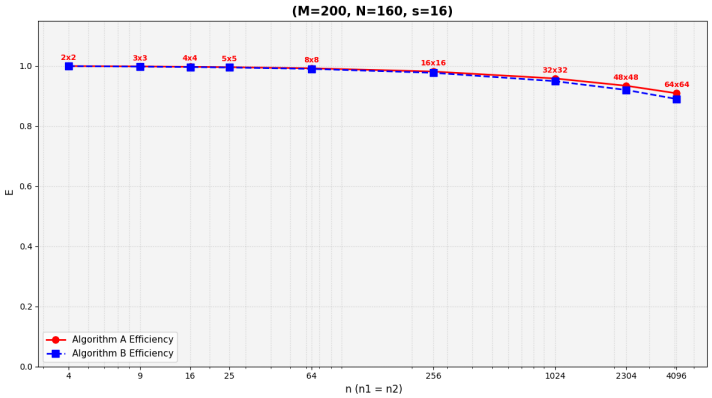
Starting from an existing arithmetic/communication cost-model framework, I adapted the expressions to the two iterative formulations studied, implemented the calculations in Python, and independently generated the three plots. The modeled efficiency is $E = T_1/(pT_p)$, with iteration count $s=16$, $C_1=10^{-9}$, $\tilde{C}_1=10^{-8}$, $C_2=10^{-10}$. The table lists predicted efficiencies for a 64×64 process grid ($p=4096$).

Matrix dimensions (M × N)	Algorithm A	Algorithm B
200 × 160	90.84%	88.94%
2,000 × 1,600	99.11%	98.94%
20,000 × 16,000	99.91%	99.89%

Under the specified model parameters, both algorithm variants have similar predicted efficiency; the model attributes less relative overhead to communication in the larger cases. These are my own **calculated model results**, not execution-time measurements with `MPI.Wtime()`.

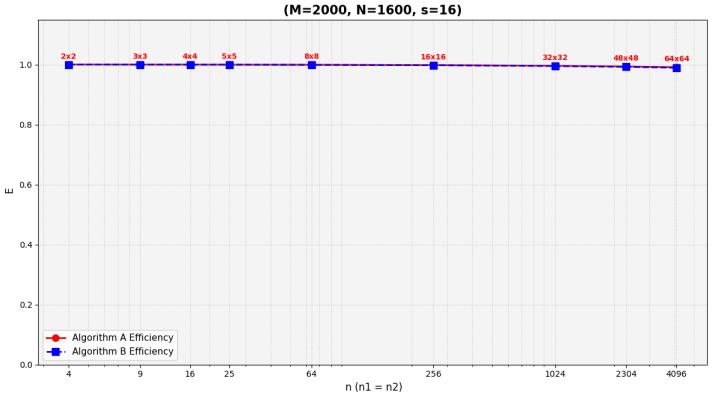
3. Efficiency curves (continued)

Figure: modeled efficiency of Algorithm A/B, M=200, N=160



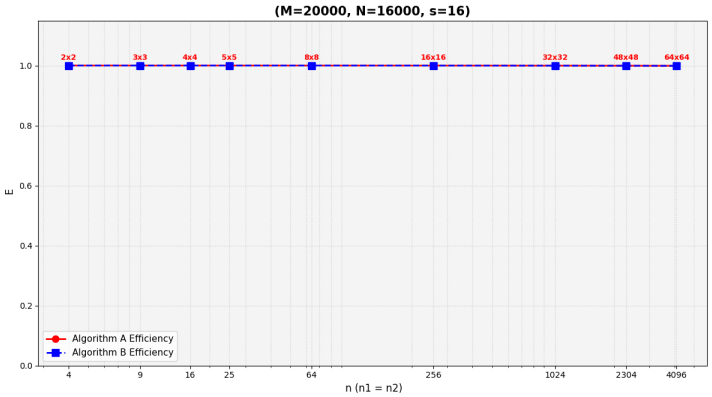
Original user-supplied image. X-axis: modeled rank count for a square process grid. Y-axis: efficiency from a parameterized cost model, not measured MPI runtime.

Figure: modeled efficiency of Algorithm A/B, M=2000, N=1600



Original user-supplied image. X-axis: modeled rank count for a square process grid. Y-axis: efficiency from a parameterized cost model, not measured MPI runtime.

Figure: modeled efficiency of Algorithm A/B, M=20000, N=16000



Original user-supplied image. X-axis: modeled rank count for a square process grid. Y-axis: efficiency from a parameterized cost model, not measured MPI runtime.

4. Current research: PCGR and preconditioned CG

Current ongoing work investigates PCGR / preconditioned conjugate-gradient methods for inverse problems. The supplied serial notebook is titled λ-PCG and implements a regularized **preconditioned CGNR (PCGNR)** variant. It uses cross approximation, QR/SVD recompression and a low-rank preconditioner. For its specific synthetic test, the notebook reports a rank-16 preconditioner and 1 PCGNR iteration versus 14 CGNR iterations under the same stopping criterion. These are iteration counts; they do not include preconditioner setup cost and do not imply a completed MPI-PCGR package or confirmed runtime speedup.

5. Other implemented numerical methods

MPI cross approximation	Distributed dense-matrix blocks, residual-based pivot search, global communication and incremental low-rank factors in a research prototype.
One-dimensional inverse problem / CGNR	Discretized forward model and synthetic noisy observations; numerical reconstruction compared with direct Tikhonov reference; preconditioning experiments.
Polynomial-parameterized reconstruction	Polynomial representation of the unknown one-dimensional model and numerical optimization-based fitting.
Earlier CG and Nesterov studies	Archived serial and distributed CG experiments. Earlier Nesterov-type accelerated iteration work is noted but source code was not provided in this file set.

6. Conclusions, evidence and next steps

My implementation record includes **Python and MPI CG-type programs, persistent-collective communication patterns, algorithm-specific cost-model studies and original graphs, MPI cross-approximation code, and numerical experiments with preconditioned regularized CGNR**. My next priorities are deterministic MPI-vs-serial validation, controlled wall-clock measurements, benchmarking standard versus persistent reductions, and continuing PCGR/preconditioning work.

A 31-slide Russian-language research seminar titled “High-performance implementations of the conjugate gradient method for ill-posed linear inverse problems” was presented at Moscow State University on May 13, 2026. This portfolio includes code, analytical-model scripts, the original plots, seminar slides and an attribution statement. No peer-reviewed publication or invention of the underlying CG methods is claimed.

Files: README.md · CONTRIBUTIONS.md · mpi/ · performance/ · notebooks/ · figures/ · presentations/

Based on supplied code, the May 2026 seminar and original figures. Prepared as an application research summary, not a peer-reviewed paper.